Budget Split Meets Geography

Heuristics for Orienteering with Service Times in Bike-Share Maintenance

Gupta Raman (A0265843W) · Koh Jun Hong Nicolas (A0272836W)
Terence Wong Wei Wen (A0273097W) · Amy Ziqi Kang (A0350502W)
CS4234 Optimisation Algorithms · AY2026/27 Semester 1

Abstract. Bike-share operators dispatch maintenance trucks to repair bikes and docks, but one shift rarely allows every station with pending work to be served, so planners must decide which stations to visit and in what order. Because each visit consumes on-site service time as well as driving time, we model this as the Orienteering Problem with fixed service times (OP-ST): a depot-returning route collects station rewards while road travel and service share one hard time budget. OP-ST couples a 0/1 knapsack (which stations) with a travelling salesperson problem (in what order); it is NP-hard and APX-hard, and the best known general guarantee, a factor of $2 + \varepsilon$, relies on machinery too heavy for routine dispatch. We therefore study simple heuristics against certified optima. Our proposed split-only heuristic reserves a fraction α of the budget for service, selects stations with an exact service knapsack, routes them by cheapest insertion and 2-opt, repairs and augments the route, and searches a 15-value α grid; a compact variant also restricts candidates to H3 hexagonal neighbourhoods. We built 540 instances from real Oslo City Bike station locations and OpenStreetMap drive times with synthetic maintenance states, and certified every optimum with a Held–Karp style subset dynamic program. Split-only achieved a mean empirical ALG/OPT ratio of 0.998, matched the optimal reward on 92.8% of instances and never fell below 0.917, whereas the worst ratios of cheapest insertion and a density-then-route baseline were 0.600 and 0.667. It needed a median of 8 ms per instance, and 18.5 ms at 16 stations, where exact solving took 2.9 s and grows exponentially. Compact pools lifted the worst ratio to 0.964 at about four times the runtime and never reduced reward. A finer reward model shows the results are not an artefact of reward ties. All ratios are empirical observations on our suite, not approximation guarantees.

1. Introduction

The Orienteering Problem (OP) takes its name from the sport in which competitors visit scored control points and must return before a time limit (Tsiligirides, 1984; Golden et al., 1987). Given locations with rewards, pairwise travel times, a start and end point and a budget, we must choose a subset of locations and an order in which to visit them so that the route fits the budget and the collected reward is maximal. Vansteenwegen et al. (2011) describe OP as "a combination between the Knapsack Problem (KP) and the Travelling Salesperson Problem (TSP)": choosing *which* locations resembles 0/1 knapsack, choosing *the order* is a TSP, and neither half can be solved alone because the "weight" of a chosen set is the length of its shortest tour, which is not additive. OP belongs to the broader family of TSPs with profits, in which travel cost is a constraint and collected profit the objective (Feillet et al., 2005).

In many applications each visit also takes time on site: a technician repairs equipment, a tourist spends an hour in a museum. Vansteenwegen et al. (2011) note that in most practical cases visiting a vertex requires service time, so the time limit bounds visiting as well as travel. We call this the Orienteering Problem with service times (OP-ST). Service time consumes the same budget as driving but does not depend on route geometry, which creates a genuine tension: time spent servicing a valuable but slow station cannot be spent reaching other stations. Orienteering-type models with visit durations appear in tourist trip design (Vansteenwegen & Van Oudheusden, 2007; Gavalas et al., 2014), planned maintenance of distributed equipment by technicians (Tang et al., 2007) and post-disaster needs assessment (Balcik, 2017); surveys catalogue many further variants (Gunawan et al., 2016; Vansteenwegen & Gunawan, 2019).

Our motivation is bike-share operations. Bikes break, docks jam and stations become full or empty, and operators dispatch vans to rebalance and repair. Operations research on these systems has concentrated on rebalancing with capacitated vehicles and station inventory targets (Chemla et al., 2013; Raviv et al., 2013; Schuijbroek et al., 2017; Laporte et al., 2018). Rainer-Harbach et al. (2015) add fixed handling times and a working-time limit per vehicle, and Ho and Szeto (2014) explicitly select a subset of stations to visit, the core idea of orienteering. Broken bikes have mostly been treated as cargo within repositioning (Wang & Szeto, 2018) or as a source of user dissatisfaction (Kaspi et al., 2017), and Oslo has served as a testbed for dynamic rebalancing (Gleditsch et al., 2024). We set inventory and capacity aside and study a simpler but

realistic decision: a single maintenance truck with a fixed shift and a list of stations with pending work of known value and duration. Which stations should it serve?

Visiting the highest-reward stations first fails whenever valuable stations are scattered. In Figure 1, two distant high-reward stations would each consume most of the budget in travel, while a cluster of moderate stations near the depot yields more total reward because the truck spends its time servicing rather than driving. Good solutions must weigh reward against service cost and *marginal* travel cost, and the latter depends on the whole route.

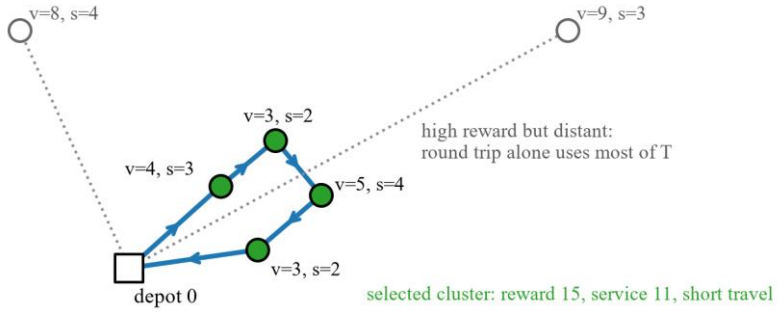


Figure 1. OP-ST in miniature. Green stations form a compact cluster served by one closed route from the square depot; the grey stations have higher individual reward v but would consume most of the budget T in travel. Values s are service times.

This project asks: **can explicit search over the service/travel budget split, supplemented by geographically compact candidate pools, produce robust near-optimal solutions on Oslo-derived maintenance instances?** We formalised OP-ST and its reduction to metric Orienteering; implemented an exact subset dynamic program (DP), three baselines and two split heuristics behind one shared validator; built a sandbox from real Oslo station locations and OpenStreetMap travel times; and evaluated everything on a 540-instance suite with certified optima, per-α traces and reward-tie enumeration. The scope is deliberately narrow (one truck and depot, deterministic symmetric travel, fixed service times, a hard budget, no capacities or time windows), which keeps the problem close to classical OP and makes exact optima possible.

2. Problem Definition and Theoretical Background

2.1 Formal OP-ST definition

An instance has a depot 0 and candidate stations $V = \{1, \dots, n\}$. Station i has reward $v_i \geq 0$ and service time $s_i \geq 0$, with $s_0 = 0$. Travel times $d_{ij} \geq 0$ on $V \cup \{0\}$ are symmetric and satisfy the triangle inequality, and T bounds total working time. A solution is a subset $S \subseteq V$ with an ordering $\pi = (\pi_1, \dots, \pi_k)$ forming the closed route $0 \to \pi_1 \to \cdots \to \pi_k \to 0$. With $\pi_0 = \pi_{k+1} = 0$, the problem is

$$\max_{S \subseteq V,\, \pi} \sum_{i \in S} v_i \quad \text{subject to} \quad \sum_{t=0}^{k} d_{\pi_t \pi_{t+1}} + \sum_{i \in S} s_i \leq T. \qquad (1)$$

The first sum in the constraint is travel time, which depends on the subset and its order; the second is service time, which depends only on the subset. Each station is visited at most once, the empty route is always feasible, and all times are integer seconds in our implementation.

2.2 Relationship to knapsack and TSP

If all travel times were zero, (1) would reduce to $\sum_{i \in S} s_i \leq T$ and OP-ST would be exactly 0/1 knapsack with values v_i and weights s_i (Kellerer et al., 2004). If instead the subset were given, checking whether some order fits would be a TSP on $S \cup \{0\}$. OP-ST couples the two: a station's effective weight is its service time plus whatever detour it adds, and the detour depends on the other chosen stations. This coupling is why selecting stations by service efficiency and only then routing them can go wrong, and it motivated the split heuristics of Section 3.

2.3 Reduction to ordinary Orienteering

Service times can be folded into edge costs. For an undirected closed route, define

$$d'_{ij} = d_{ij} + \frac{s_i + s_j}{2} \quad \text{for } i \neq j, \qquad d'_{ii} = 0. \qquad (2)$$

Every visited station has exactly two incident route edges, each contributing half its service time, so its service is counted exactly once; the depot contributes nothing. Hence the transformed cost of any closed route equals its travel time plus its total service time, and OP-ST is ordinary Orienteering with costs d'. Metricity is preserved: $d'_{ik} \le d_{ij} + d_{jk} + (s_i + s_k)/2 \le d'_{ij} + d'_{jk}$ because $s_j \ge 0$. We derived this half-sum form ourselves; it is equivalent to the "leaf extension" of Angelelli et al. (2014), which attaches a pendant edge of length $s_i/2$ to each customer. Our exact solver stores $2d'_{ij} = 2d_{ij} + s_i + s_j$ and compares against $2T$, so odd service times never need rounding; rounding a half-second down could otherwise declare an infeasible route feasible.

2.4 Complexity and approximation context

Orienteering is NP-hard (Golden et al., 1987, as reported by Vansteenwegen et al., 2011). Directly: giving every vertex reward 1 and asking whether all n vertices fit within budget is the TSP decision problem, and setting every $s_i = 0$ embeds this in OP-ST. Orienteering is also APX-hard, so no polynomial-time approximation scheme exists for general metrics unless P = NP (Blum et al., 2007). For general metrics, Blum et al. (2007) gave the first constant factor, 4 for rooted paths, through the min-excess path problem; Bansal et al. (2004) improved this to 3 for point-to-point orienteering; and Chekuri, Korula and Pál (2012) achieved $2 + \varepsilon$ for point-to-point orienteering in undirected graphs, which covers closed tours by setting start equal to end and remains the best known general bound. A $(2+\varepsilon)$-approximation returns a feasible route with $\mathrm{ALG} \ge \mathrm{OPT}/(2 + \varepsilon)$ on *every* instance, and because the transformation of Section 2.3 preserves metricity, the guarantee transfers to closed-route OP-ST.

We treat these results as context only and keep two notions strictly apart. A *theoretical guarantee* is a proven bound for all instances. An *empirical ratio* $\mathrm{ALG}/\mathrm{OPT}$ is a measurement on a particular suite. Every ratio in this report is of the second kind: a minimum observed ratio of 0.96 means only that no instance in our suite did worse.

3. Algorithms

All six algorithms read one shared instance representation (depot, station records with reward, service time and H3 cell, and an integer travel-time matrix) and return one shared solution representation. We did not trust an algorithm's own totals: a shared validator recomputes closure at the depot, uniqueness of visits, reward, travel, service and total time, and rejects any route exceeding T. All 3,240 final solutions of the main suite passed. Table 1 summarises the algorithms.

Table 1. Algorithms compared. Pseudo-polynomial means polynomial in n and the service budget in seconds.

Algorithm	Central idea	Uses geography	Exact	Running time	Role
Exact DP	Subset DP over (subset, endpoint) states	Yes	Yes	$O(n^2 2^n)$	OPT
Best single	Best station reachable alone	Minimal	No	$O(n)$	Floor
Density then route	Greedy reward/service at α = 0.7, then route	After selection	No	Polynomial	Baseline
Cheapest insertion	Insert by reward per marginal total time	Yes	No	Polynomial	Baseline
Split-only	Knapsack per α, route, repair, augment	After selection	No	Pseudo-poly.	Main
Split + compact	Split-only over global and H3 pools	Yes	No	Pseudo-poly.	Main

3.1 Exact subset dynamic programming

The exact solver is a Held–Karp style DP (Bellman, 1962; Held & Karp, 1962) on the transformed costs. For a subset S, encoded as an n-bit mask, and endpoint $j \in S$, let $C(S, j)$ be the minimum transformed cost of a path from the depot visiting each station of S once and ending at j. Then $C(\{j\}, j) = d'_{0j}$ and

$$C(S, j) = \min_{i \in S \setminus \{j\}} \left[C(S \setminus \{j\}, i) + d'_{ij} \right]. \qquad (3)$$

A subset is feasible if $\min_{j \in S} [C(S, j) + d'_{j0}] \le T$, and OPT is the largest reward over feasible subsets. A predecessor table recovers the route; ties prefer higher reward, then lower total time, then the lexicographically smallest route. Correctness follows by induction on $|S|$: any path through S ending at j arrives from some last station i, and its prefix is a path through $S \setminus \{j\}$ ending at i, so recurrence (3), which tries every i with the cheapest prefix, is exact; adding the closing edge tests feasibility exactly. There are $n2^n$ states with up to n predecessors each, giving $O(n^2 2^n)$ time and $O(n2^n)$ memory, so we limited exact

solving to $n \leq 16$. We cross-checked the DP against brute-force permutation enumeration on random metric instances with up to eight stations.

Exact DP was essential: certified optima turn every result into a statement about distance from the true optimum, not merely a comparison between heuristics.

3.2 Best-single baseline

Best single serves the highest-reward station i with $2d_{0i} + s_i \leq T$, in $O(n)$ time. It is a sanity floor showing whether instances genuinely require multi-station routes; the generator guarantees it is never empty.

3.3 Service density then route

This baseline reflects a fixed planning split. It reserves $0.7T$ for service, adds stations greedily by v_i/s_i while service fits, and routes them by cheapest insertion with 2-opt improvement (Croes, 1958). If the route exceeds T, repair repeatedly removes the station with the smallest reward per second saved; if time remains, augmentation inserts stations by the largest reward per marginal second. Geography is ignored during selection, so it can commit to service-efficient but scattered stations.

3.4 Cheapest insertion

This baseline is geography-aware from the start: for every unrouted station and position it computes the marginal total time $\Delta = d_{ai} + d_{ib} - d_{ab} + s_i$ of inserting station i between route neighbours a and b, inserts the feasible station with largest v_i/Δ, applies 2-opt, and stops when nothing fits. For full TSP tours, cheapest insertion is within a factor 2 of optimal under the triangle inequality (Rosenkrantz et al., 1977), but that bound does not transfer to our selective setting, and the method is myopic: it never removes a station once inserted.

3.5 Split-only grid heuristic

The central heuristic separates the two uses of the budget. For a service fraction α it reserves $B_s = \lfloor \alpha T \rfloor$ seconds for service and leaves the rest for travel, then follows Algorithm 1. Selection solves the 0/1 knapsack exactly by the textbook DP over used service time in $O(nB_s)$ (Kellerer et al., 2004), which is pseudo-polynomial but fast for budgets of a few thousand seconds. Routing, repair and augmentation reuse the baseline procedures.

Algorithm 1. Split-only grid heuristic.

```
best <- empty route (0, 0)
for alpha in (0.20, 0.25, ..., 0.90):
    S     <- exact 0/1 knapsack: values v, weights s, capacity floor(alpha * T)
    route <- cheapest insertion of S, then 2-opt
    route <- repair: while total(route) > T, remove min v_i / (time saved)
    route <- augment: insert max v_i / (marginal time) while total(route) <= T
    validate(route); keep if (reward, -total time, route) beats best
return best
```

We initially considered binary search under an assumed monotone relationship between the service allocation and the resulting solution. However, changing α discontinuously changes the selected subset, routing and repair decisions, so neither feasibility quality nor reward is monotone. The traces confirmed this: split-only reward changed with α on 447 of 540 instances and was non-monotone on 427 (Section 6.4). We therefore replaced binary search with an explicit 15-value grid, $\alpha \in \{0.20, 0.25, \ldots, 0.90\}$; full pseudocode is in Appendix A.

3.6 Split plus compactness

The service knapsack is blind to geography: it may pick efficient but mutually distant stations and leave repair to discard some. We therefore added candidate pools that are compact by construction. Each station is labelled with its H3 hexagonal cell at resolution 8 (Brodsky, 2018), whose cells average under one square kilometre; the 40 sandbox stations occupy nine cells. For every occupied cell we form the pool of stations in that cell and the pool in that cell plus its six neighbours, remove duplicates, and always keep the global pool. Algorithm 1 runs on every pool and α with augmentation restricted to the pool, and the best validated solution is returned. H3 only groups stations; travel times still come from the road matrix.

Retaining the global pool was deliberate. Compact pools are a hedge that helps when the best route is local but could hurt if it replaced global search. Because the global pool is evaluated with exactly the split-only procedure, split+compact maximises over a superset of split-only's candidates and can never obtain lower

reward on any instance. This dominance is a property of our implementation, not an approximation theorem. The cost is runtime proportional to the number of distinct pools: 10.2 per instance on average (range 3–19), namely the global pool, 5.3 single-cell pools and 3.9 one-ring pools that survive deduplication, so each instance evaluates about 150 pool–α candidates.

4. Oslo Sandbox and Instance Generation

4.1 Why Oslo

We needed real station geography with an openly licensed feed and enough spatial variety for both clustered and spread-out instances. Oslo City Bike publishes station information in the General Bikeshare Feed Specification (GBFS) v2.3 under the Norwegian Licence for Open Government Data (Oslo Bysykkel, 2026; MobilityData, 2022). Its compact centre keeps exact optima tractable while preserving realistic variation in station separation. We had no access to operator maintenance records and do not claim to model Oslo's actual operations.

4.2 Real data components

We fetched the station-information feed once, sending the identifying client header the operator asks for, and cached the raw JSON with its retrieval time and SHA-256 checksum; it lists 267 stations. We kept the 40 stations nearest the centroid of all station coordinates and designated the most central one as an experimental depot, leaving 39 candidates. We downloaded the OpenStreetMap drive network around them with OSMnx (OpenStreetMap contributors, 2026; Boeing, 2017). Edge speeds come from speed-limit tags, with missing values imputed as the mean limit of the same road type, and edge times are free-flow, ignoring congestion and turn delays. Snapping stations to their nearest graph nodes and running Dijkstra's algorithm (Dijkstra, 1959) in NetworkX (Hagberg et al., 2008) gave a directed 40×40 matrix of shortest-path times.

One-way streets make that matrix asymmetric: 96.9% of pairs differ between directions, by a median of 17 seconds. Because our model and the reduction of Section 2.3 assume symmetry, we used $\lceil (d_{ij} + d_{ji})/2 \rceil$, rounding up so the symmetrized average is not understated. The average may still be below one of the two original directional times. The result is finite, non-negative and symmetric with zero diagonal, and an exhaustive check over all triples found no triangle-inequality violations. Pairwise times range from 0 to 320 seconds (median 146). Two station pairs snap to the same road node and so have zero travel time.

4.3 Synthetic maintenance states

No public data says which Oslo stations need maintenance or how long jobs take, so we combine real road structure with a transparent seeded generator. Synthetic states are modelling assumptions, not observations. Each station draws a task count $k_i = max(1, \text{Poisson}(2))$ and urgency $u_i \in \{1,2,3\}$ uniformly. Service time is $s_i = \lceil (90 + 120k_i)\sigma \rceil$ seconds (90 s set-up, 120 s per task, service scale σ), and reward is $v_i = k_i u_i$. In the outlier regime the most isolated station's reward is quadrupled, creating a tempting but distant target. Every instance stores its seed and full data as JSON; Appendix B details the data pipeline and generator.

4.4 Spatial regimes

Each instance selects n candidates under one of four regimes, each designed to expose a different weakness. **Compact**: an anchor station and its $n - 1$ nearest neighbours, so travel is cheap and the problem is mostly a knapsack. **Dispersed**: farthest-point selection from the anchor, so travel dominates and geography-blind selection should suffer. **Mixed**: 70% clustered stations plus farthest-point fill, mimicking a busy neighbourhood with scattered requests. **Outlier**: $n - 1$ clustered stations plus the farthest station with quadrupled reward, testing whether a method is lured into a costly detour.

4.5 Budget and service variation

Budgets scale with each instance's geometry. We route all n stations by cheapest insertion with 2-opt and set T to 35% (tight), 50% (medium) or 65% (loose) of that route's total time, so selection always matters; T is raised if needed so that at least one station is individually feasible. Service scales $\sigma \in \{0.5,1,2\}$ shift instances between travel- and service-dominated. Table 2 gives the resulting $3 \times 4 \times 3 \times 3 \times 5 = 540$ instances, frozen once generated.

Table 2. Factorial design of the primary exact-comparison suite.

Factor	Levels	Purpose
Size n	8, 12, 16	Range where exact DP certifies OPT
Spatial regime	compact, dispersed, mixed, outlier	Vary travel structure and decoys
Service scale σ	0.5, 1.0, 2.0	Balance between service and travel
Budget	35%, 50%, 65% of full-route time	Tight, medium, loose pressure
Seed	1–5	Replications per design cell
Total	540 instances	3,240 algorithm runs

5. Experimental Methodology

Every algorithm ran on every instance, giving 3,240 results stored with routes, times, parameters and validator verdicts. We also stored 16,200 α-level trace rows and 82,350 pool-level rows recording every compact pool at every α. All randomness used recorded seeds and all ties are broken deterministically; when we added instrumentation, rerunning the suite reproduced the earlier reward summaries for all six algorithms.

OPT is the validated exact DP reward. For each heuristic solution we report the reward ratio $\mathrm{ALG}/\mathrm{OPT} \in [0,1]$ and summarise it by mean, median, tenth percentile (P10) and minimum; the minimum is the worst case observed, not a guarantee. Because many ratios equal exactly 1, we also report the exact-reward match rate. Matching OPT's reward differs from reproducing the exact solver's solution, since several subsets or orders can share the optimal reward, so we also record canonical-subset matches and route matches up to reversal. Runtime is wall-clock milliseconds per instance on one laptop process.

Because all algorithms share instances, we use paired comparisons: for an ablation from A to B we count instances where B's ratio is higher, equal or lower. Pairing cancels per-instance difficulty and exposes rare losses that averages hide. The two ablations are density-then-route to split-only, isolating split search, and split-only to split+compact, isolating compact pools.

Because median and P10 ratios of both split heuristics were exactly 1, we checked whether coarse integer rewards create ties that make OPT's value easy to hit. We enumerated, from the DP tables, every feasible subset attaining the optimal reward, and built a 72-instance sensitivity suite ($n \in \{8,12\}$, four regimes, three budgets, three seeds, $\sigma = 1$) with the finer reward $v_i = 10k_i + u_i$. This suite is a robustness check, not a replacement for the primary design.

6. Results

6.1 Overall quality and runtime

Table 3. Results on the 540-instance exact suite. Ratios are ALG/OPT; exact match is the share of instances with reward equal to OPT; runtime is the median per instance.

Algorithm	Mean	Median	P10	Min	Exact match	Runtime (ms)
Best single	0.384	0.333	0.205	0.138	0.9%	0.06
Density then route	0.966	1.000	0.892	0.667	53.1%	0.31
Cheapest insertion	0.986	1.000	0.946	0.600	71.7%	0.35
Split-only	0.998	1.000	1.000	0.917	92.8%	7.97
Split + compact	0.999	1.000	1.000	0.964	95.2%	31.2
Exact DP	1.000	1.000	1.000	1.000	100%	110.7

Table 3 and Figure 2 summarise all 540 instances. Best single averaged only 38% of OPT, confirming that instances need multi-station routes. Density-then-route and cheapest insertion look strong on average but have weak tails: the former matched OPT on only half the instances and fell to two-thirds of OPT, and the latter matched on 72% but produced the single worst multi-station result, 0.600, on a small tight mixed-regime instance that split-only solved optimally. Split-only, our proposed method, stands out: its mean ratio of 0.998 means it lost on average 0.2% of the optimal reward, its P10 of 1 means at least nine instances in ten were solved to optimal reward, and even its worst instance collected 91.7% of OPT. Split+compact tightened this further, to 95.2% matches and a worst case of 0.964.

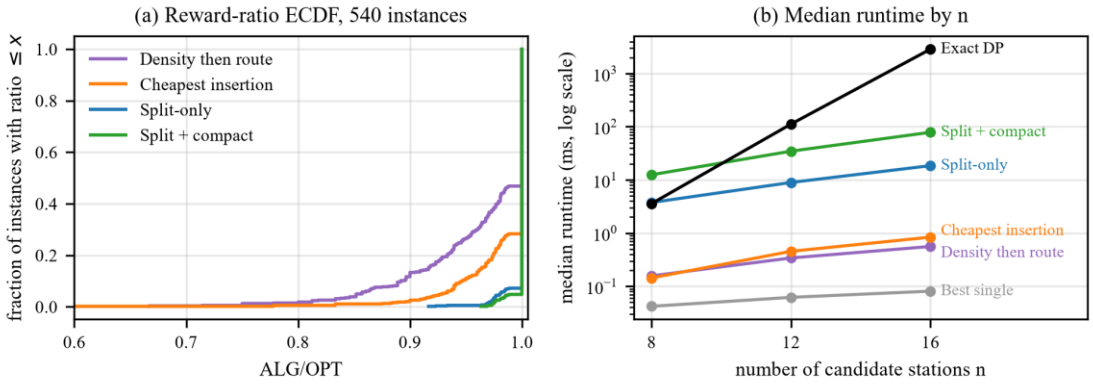


Figure 2. (a) Empirical cumulative distribution of ALG/OPT from 0.6; curves further right are better, and the jump at 1 equals the exact-match rate. Best single (mean 0.38) is omitted. (b) Median runtime per instance against n, log scale.

The price is modest. Split-only took a median 8 ms, about 14 times faster than exact DP overall and 155 times faster at $n = 16$ (18.5 ms against 2.9 s), and split+compact took 31 ms, because each repeats knapsack and routing 15 times per pool. Exact DP was faster than split+compact at $n = 8$ (3.6 against 12.5 ms), but grew from 3.6 ms to 112 ms to 2.9 s as n rose from 8 to 12 to 16, about 30 times per four added stations, as $n^2 2^n$ predicts. This is the practical reason exact comparison stops at $n = 16$; the heuristics grew only five- to six-fold over the same range.

6.2 Paired ablations

From density-then-route to split-only, 229 instances improved, 311 tied and none worsened, with mean gain +0.0325. Split-only therefore improved 42% of instances and was never worse on any of them, while adding only milliseconds of computation.

From split-only to split+compact, 13 instances improved, 527 tied and none worsened, with mean gain only +0.0009. No losses is guaranteed by the global pool; few gains is an empirical finding. Compactness is an incremental tail improvement, not the main source of performance, but it acts where robustness matters: the 13 improvements include split-only's three worst instances, all raised to OPT, lifting the worst ratio from 0.917 to 0.964. All 13 came from cell-plus-ring pools, and all occurred under tight (nine) or medium (four) budgets, never loose ones. Plausibly, geographic focus matters most when the budget leaves no slack for detours, though our traces do not establish this causally.

6.3 Spatial-regime behaviour

Table 4 gives the worst ratio per regime. For split-only the hardest regime was compact (0.917), then mixed, outlier and dispersed (0.964). We had expected dispersed instances to be hardest for geography-blind selection; one plausible but untested explanation is that in compact instances many combinations cost about the same, so a wrong choice under a tight budget loses reward without any large detour signalling the mistake. Compact pools raised the compact, mixed and outlier minima to 0.970–0.974 but left the dispersed minimum at 0.964, consistent with dispersed instances offering few dense neighbourhoods to exploit. Greedy baselines showed the reverse pattern, with worst cases in dispersed and mixed instances and the outlier regime relatively easy. Appendix C breaks results down by budget and service scale.

Table 4. Worst observed ALG/OPT within each spatial regime (135 instances each).

Algorithm	Compact	Dispersed	Mixed	Outlier
Density then route	0.800	0.667	0.714	0.864
Cheapest insertion	0.882	0.778	0.600	0.912
Split-only	0.917	0.964	0.923	0.933
Split + compact	0.973	0.964	0.970	0.974

6.4 Alpha behaviour

Figure 3(a) shows how the grid works. At small α the knapsack picks few stations and augmentation fills the remaining time, adding 3.9 stations on average at α = 0.2; at large α it picks too many and repair removes 1.4 on average at α = 0.9. Each α therefore proposes a different starting subset, and the best proposal differs between instances. The best single fixed α achieved 73.5% exact matches, while the full grid achieved 92.8%.

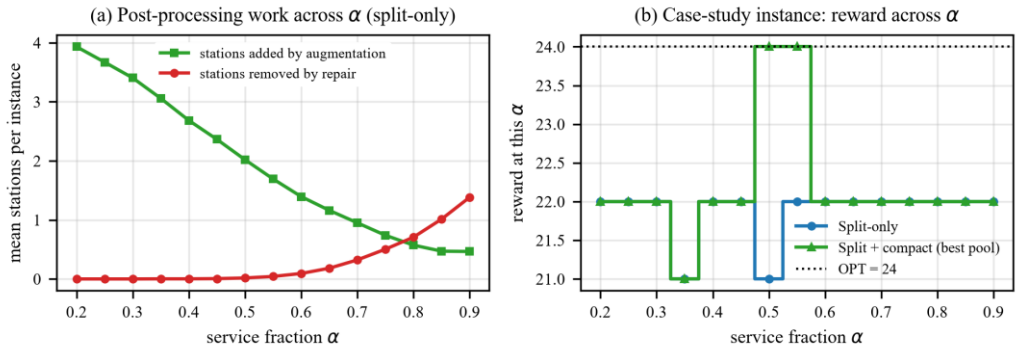


Figure 3. (a) Mean number of stations that augmentation adds and repair removes at each α for split-only, over all 540 instances. (b) Reward at each α on the case-study instance of Section 6.7: both curves are step functions that rise and fall, so binary search over α has no basis.

Figure 3(b) shows a typical profile: split-only reward moves between 21 and 22, dropping at α = 0.35 and 0.5, while the compact variant reaches OPT only at α ∈ {0.50, 0.55}. Reward was non-monotone in α on 427 of 540 instances. The reported "chosen" α should not be over-read: because ties keep the first value found, it was the grid's lower end, 0.20, on 309 instances, yet on average 10 of the 15 values achieved the same final reward, and only 27 instances had a unique best α. Appendix D gives further traces and per-α breakdowns.

6.5 Solution similarity and reward ties

Split+compact obtained the optimal reward on 514 instances. On 485 it chose the same subset as the exact solver, on 406 also the same order up to reversal; on 79 it served that subset in another order, and on 29 it reached OPT with a different subset. These are not failures: any route with reward OPT is equally optimal. Enumeration showed that ties are common but not overwhelming: instances had a mean of 2.46 and median of 1 optimal subsets, and 47.6% had more than one. Ties explain why subset and route matches lag reward matches, but most instances had a single optimal subset that the heuristic had to find.

6.6 Reward sensitivity

Under fine rewards, the mean number of optimal subsets fell to 1.76 and the share with multiple optima to 38.9%. Both split heuristics still matched OPT on 93.1% of the 72 instances, with worst cases 0.966 (split-only) and 0.978 (split+compact), against 0.821 for density-then-route and 0.879 for cheapest insertion, and the ordering of methods was unchanged. Coarse rewards contribute to the subset and route differences of Section 6.5 but do not explain the split heuristics' performance.

6.7 Route case study

Figure 4 shows the instance with the largest compactness gain: compact regime, $n = 12$, σ = 0.5, tight budget 1,168 s, OPT = 24. The exact solver serves stations with rewards 4, 8 and 12, using 1,164 s. Split-only replaces the reward-8 station with a reward-6 station nearer the depot, scoring 22 with 104 s unused. The traces explain why: in the global pool the knapsack kept preferring cheap-to-service stations in a southern cluster, so every α yielded 21 or 22 after routing and repair. Restricting candidates to the one-ring H3 neighbourhood of the reward-8 station removed those distractors; at α = 0.5 the knapsack chose the reward-8 and reward-12 stations and augmentation added the reward-4 station, reproducing the optimum exactly. Appendix E shows four further cases in our solution explorer.

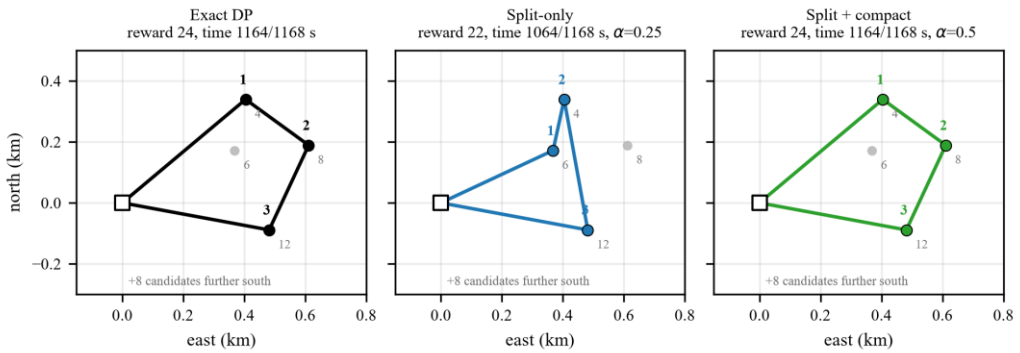


Figure 4. Case study of Section 6.7. Small numbers are rewards; bold numbers give visit order from the square depot. Split-only misses the reward-8 station; split+compact recovers the optimal route. Lines are schematic; times come from the road matrix.

7. Discussion and Limitations

Split search works because service and travel compete for one budget and the right balance differs between instances: service-heavy instances want a large α, travel-dominated ones a small α so the route can reach enough stations. Enumerating 15 allocations is a cheap hedge against committing to one assumption about that balance, and the knapsack need not be accurate about travel because repair removes stations with expensive detours and augmentation fills leftover time.

Compactness added modest average value because global split search already matched OPT on 92.8% of instances, and since the global pool is retained, compact pools only matter where the global knapsack is consistently distracted, as in Figure 4.

Our conclusions have clear limits. Exact comparison covers only $n \leq 16$, and ratios on larger instances may be worse. Maintenance states are synthetic, so results describe our generator's distribution, not Oslo's real operations. The model has one truck, no capacities or time windows, and deterministic, symmetrised free-flow travel. The central sandbox is small, with a median inter-station time of about two and a half minutes, so service often dominates travel and instances may be relatively easy. We did not implement the $(2+\varepsilon)$ algorithm, so we cannot compare against it. Natural extensions are larger suites benchmarked against integer-programming solvers, rewards calibrated on observed station-status history, and multiple vehicles with time windows.

8. Conclusion

We asked whether searching the service/travel budget split, supplemented by compact geographic candidate pools, yields robust near-optimal solutions on Oslo-derived maintenance instances. On our suite the answer is yes, and the proposed split-only heuristic carries most of the weight. Across 540 instances with certified optima it achieved a mean empirical ALG/OPT ratio of 0.998, matched the optimal reward on 92.8% of instances and never collected less than 91.7% of OPT. The simple baselines it builds on were fast but fragile, with worst ratios of 0.600 for cheapest insertion and 0.667 for density-then-route, and split-only improved on the latter on 229 instances without losing on any. These gains cost little: a median of 8 ms per instance and 18.5 ms at 16 stations, against 2.9 s for exact DP, whose runtime multiplied by about 30 for every four added stations while split-only's grew only five-fold from 8 to 16 stations. Split-only therefore delivered near-exact quality at a small fraction of the exact solver's cost.

Compact H3 pools are a complementary safeguard rather than the main source of quality. They improved 13 hard instances, including split-only's three worst, lifting the worst ratio to 0.964 at roughly four times the runtime, and by construction they never reduced reward. Exact DP made these statements trustworthy, the shared validator made every route checkable, and the results held under a finer reward model with fewer ties.

These remain empirical observations on generated instances with at most 16 stations and synthetic maintenance states, not approximation guarantees. Within those limits, the project shows that a simple and transparent idea, searching over how a shift's time is divided between service and travel, can be carried onto real geographic data in a reproducible, fully validated pipeline, and that it comes remarkably close to optimal at a cost small enough for routine planning.

References

Angelelli, E., Bazgan, C., Speranza, M. G., & Tuza, Z. (2014). Complexity and approximation for traveling salesman problems with profits. *Theoretical Computer Science*, 531, 54–65. https://doi.org/10.1016/j.tcs.2014.02.046

Balcik, B. (2017). Site selection and vehicle routing for post-disaster rapid needs assessment. *Transportation Research Part E*, 101, 30–58. https://doi.org/10.1016/j.tre.2017.01.002

Bansal, N., Blum, A., Chawla, S., & Meyerson, A. (2004). Approximation algorithms for deadline-TSP and vehicle routing with time-windows. In *Proceedings of the 36th ACM Symposium on Theory of Computing* (pp. 166–174). https://doi.org/10.1145/1007352.1007385

Bellman, R. (1962). Dynamic programming treatment of the travelling salesman problem. *Journal of the ACM*, 9(1), 61–63. https://doi.org/10.1145/321105.321111

Blum, A., Chawla, S., Karger, D. R., Lane, T., Meyerson, A., & Minkoff, M. (2007). Approximation algorithms for orienteering and discounted-reward TSP. *SIAM Journal on Computing*, 37(2), 653–670. https://doi.org/10.1137/050645464

Boeing, G. (2017). OSMnx: New methods for acquiring, constructing, analyzing, and visualizing complex street networks. *Computers, Environment and Urban Systems*, 65, 126–139. https://doi.org/10.1016/j.compenvurbsys.2017.05.004

Brodsky, I. (2018, June 27). H3: Uber's hexagonal hierarchical spatial index. *Uber Engineering Blog*. https://www.uber.com/blog/h3/

Chekuri, C., Korula, N., & Pál, M. (2012). Improved algorithms for orienteering and related problems. *ACM Transactions on Algorithms*, 8(3), Article 23. https://doi.org/10.1145/2229163.2229167

Chemla, D., Meunier, F., & Wolfler Calvo, R. (2013). Bike sharing systems: Solving the static rebalancing problem. *Discrete Optimization*, 10(2), 120–146. https://doi.org/10.1016/j.disopt.2012.11.005

Croes, G. A. (1958). A method for solving traveling-salesman problems. *Operations Research*, 6(6), 791–812. https://doi.org/10.1287/opre.6.6.791

Dijkstra, E. W. (1959). A note on two problems in connexion with graphs. *Numerische Mathematik*, 1(1), 269–271. https://doi.org/10.1007/BF01386390

Feillet, D., Dejax, P., & Gendreau, M. (2005). Traveling salesman problems with profits. *Transportation Science*, 39(2), 188–205. https://doi.org/10.1287/trsc.1030.0079

Gavalas, D., Konstantopoulos, C., Mastakas, K., & Pantziou, G. (2014). A survey on algorithmic approaches for solving tourist trip design problems. *Journal of Heuristics*, 20(3), 291–328. https://doi.org/10.1007/s10732-014-9242-5

Gleditsch, M. D., Hagen, K., Andersson, H., Bakker, S. J., & Fagerholt, K. (2024). A column generation heuristic for the dynamic bicycle rebalancing problem. *European Journal of Operational Research*, 317(3), 762–775. https://doi.org/10.1016/j.ejor.2022.07.004

Golden, B. L., Levy, L., & Vohra, R. (1987). The orienteering problem. *Naval Research Logistics*, 34(3), 307–318. https://doi.org/10.1002/1520-6750(198706)34:3<307::AID-NAV3220340302>3.0.CO;2-D

Gunawan, A., Lau, H. C., & Vansteenwegen, P. (2016). Orienteering problem: A survey of recent variants, solution approaches and applications. *European Journal of Operational Research*, 255(2), 315–332. https://doi.org/10.1016/j.ejor.2016.04.059

Hagberg, A. A., Schult, D. A., & Swart, P. J. (2008). Exploring network structure, dynamics, and function using NetworkX. In *Proceedings of the 7th Python in Science Conference* (pp. 11–15). https://doi.org/10.25080/TCWV9851

Held, M., & Karp, R. M. (1962). A dynamic programming approach to sequencing problems. *Journal of the Society for Industrial and Applied Mathematics*, 10(1), 196–210. https://doi.org/10.1137/0110015

Ho, S. C., & Szeto, W. Y. (2014). Solving a static repositioning problem in bike-sharing systems using iterated tabu search. *Transportation Research Part E*, 69, 180–198. https://doi.org/10.1016/j.tre.2014.05.017

Kaspi, M., Raviv, T., & Tzur, M. (2017). Bike-sharing systems: User dissatisfaction in the presence of unusable bicycles. *IISE Transactions*, 49(2), 144–158. https://doi.org/10.1080/0740817X.2016.1224960

Kellerer, H., Pferschy, U., & Pisinger, D. (2004). *Knapsack Problems*. Springer. https://doi.org/10.1007/978-3-540-24777-7

Laporte, G., Meunier, F., & Wolfler Calvo, R. (2018). Shared mobility systems: An updated survey. *Annals of Operations Research*, 271(1), 105–126. https://doi.org/10.1007/s10479-018-3076-8

MobilityData. (2022). *General Bikeshare Feed Specification (GBFS), version 2.3*. https://github.com/MobilityData/gbfs/blob/v2.3/gbfs.md

OpenStreetMap contributors. (2026). *OpenStreetMap* [Road network data, Open Database License 1.0]. Retrieved November 2026, from https://www.openstreetmap.org

Oslo Bysykkel. (2026). *Realtime data* [GBFS station information, Norwegian Licence for Open Government Data 2.0]. Retrieved 9 November 2026, from https://oslobysykkel.no/en/open-data/realtime

Rainer-Harbach, M., Papazek, P., Raidl, G. R., Hu, B., & Kloimüllner, C. (2015). PILOT, GRASP, and VNS approaches for the static balancing of bicycle sharing systems. *Journal of Global Optimization*, 63(3), 597–629. https://doi.org/10.1007/s10898-014-0147-5

Raviv, T., Tzur, M., & Forma, I. A. (2013). Static repositioning in a bike-sharing system: Models and solution approaches. *EURO Journal on Transportation and Logistics*, 2(3), 187–229. https://doi.org/10.1007/s13676-012-0017-6

Rosenkrantz, D. J., Stearns, R. E., & Lewis, P. M., II. (1977). An analysis of several heuristics for the traveling salesman problem. *SIAM Journal on Computing*, 6(3), 563–581. https://doi.org/10.1137/0206041

Schuijbroek, J., Hampshire, R. C., & van Hoeve, W.-J. (2017). Inventory rebalancing and vehicle routing in bike sharing systems. *European Journal of Operational Research*, 257(3), 992–1004. https://doi.org/10.1016/j.ejor.2016.08.029

Tang, H., Miller-Hooks, E., & Tomastik, R. (2007). Scheduling technicians for planned maintenance of geographically distributed equipment. *Transportation Research Part E*, 43(5), 591–609. https://doi.org/10.1016/j.tre.2006.03.004

Tsiligirides, T. (1984). Heuristic methods applied to orienteering. *Journal of the Operational Research Society*, 35(9), 797–809. https://doi.org/10.1057/jors.1984.162

Vansteenwegen, P., & Gunawan, A. (2019). *Orienteering Problems: Models and Algorithms for Vehicle Routing Problems with Profits*. Springer. https://doi.org/10.1007/978-3-030-29746-6

Vansteenwegen, P., Souffriau, W., & Van Oudheusden, D. (2011). The orienteering problem: A survey. *European Journal of Operational Research*, 209(1), 1–10. https://doi.org/10.1016/j.ejor.2010.03.045

Vansteenwegen, P., & Van Oudheusden, D. (2007). The mobile tourist guide: An OR opportunity. *OR Insight*, 20(3), 21–27. https://doi.org/10.1057/ori.2007.17

Wang, Y., & Szeto, W. Y. (2018). Static green repositioning in bike sharing systems with broken bikes. *Transportation Research Part D*, 65, 438–457. https://doi.org/10.1016/j.trd.2018.09.016

Appendix A — Pseudocode of the Split Heuristics

The pseudocode below follows the implementation line by line. Route positions are indexed from the depot, a route is a sequence $(0, \pi_1, \ldots, \pi_k, 0)$, and total(route) is travel plus service time in integer seconds. Every tie is broken deterministically, so repeated runs return identical routes.

Algorithm A1. Shared subroutines.

```
KNAPSACK(P, Bs):                        // exact 0/1 knapsack on service time
    table <- {0: (reward 0, set {})}    // key = service seconds used
    for i in P in increasing index order:
        for each (u, (r, S)) in a snapshot of table:
            if u + s_i <= Bs:
                offer (r + v_i, S + {i}) at key u + s_i
                keep it if reward is higher (tie: smaller index sequence)
    return S of the entry with max reward, then least service used

ROUTE(S):                               // cheapest insertion, then 2-opt
    route <- (0, 0)
    while S has uninserted stations:
        insert the station and position with least marginal time
            delta = d(a,i) + d(i,b) - d(a,b) + s_i   (tie: lower index)
    return TWO_OPT(route)

TWO_OPT(route):
    repeat until no change:
        apply the first segment reversal that lowers travel time
        (equal travel accepted only if the route is lexicographically smaller)

REPAIR(route):                          // restore feasibility
    while total(route) > T and route is non-empty:
        remove the station with least v_i / (time saved by removing it)
        route <- TWO_OPT(route)

AUGMENT(route, P):                      // fill leftover time from pool P
    loop:
        F <- stations of P not on route whose cheapest insertion fits in T
        if F is empty: stop
        insert the station of F with max v_i / delta at its best position
        route <- TWO_OPT(route)
    return route
```

Algorithm A2. Split-only grid (the proposed method).

```
SPLIT_ONLY(instance, grid = (0.20, 0.25, ..., 0.90)):
    best <- empty route (0, 0)
    for alpha in grid:
        Bs    <- floor(alpha * T)
        S     <- KNAPSACK(all stations, Bs)
        route <- AUGMENT(REPAIR(ROUTE(S)), all stations)
        VALIDATE(route)                     // shared validator, strict
        if key(route) > key(best): best <- route
    return best
key(route) = (reward, -total time, route order)   // compared lexicographically
```

Algorithm A3. Split plus compactness.

```
POOLS(instance):
    pools <- [all stations]
    for each occupied H3 resolution-8 cell c, in sorted order:
        append stations whose cell is c
        append stations whose cell is c or one of its six neighbours
    drop empty pools and pools with an identical station set
    return pools

SPLIT_COMPACT(instance, grid):
    best <- empty route (0, 0)
    for P in POOLS(instance):
        for alpha in grid:
            S     <- KNAPSACK(P, floor(alpha * T))
            route <- AUGMENT(REPAIR(ROUTE(S)), P)   // augmentation stays in P
            VALIDATE(route)
            if key(route) > key(best): best <- route
    return best
```

Three implementation details matter for correctness. First, the knapsack treats service time as the only weight, so its subset may be infeasible once travel is added; REPAIR is what guarantees feasibility, and the validator re-checks every candidate rather than trusting the heuristic's own bookkeeping. Second, stations whose removal saves no time but carry positive reward receive an infinite score in REPAIR and are removed last, and stations insertable at zero marginal time receive an infinite score in AUGMENT and are inserted first. Third, because the global pool in POOLS is evaluated exactly as in SPLIT_ONLY, split plus compactness can never return lower reward than split-only on the same instance.

Appendix B — Data Pipeline and Instance Generation

Station snapshot. The GBFS station-information file was retrieved on 9 November 2026 at 16:32 UTC from the Oslo Bysykkel feed (GBFS v2.3, NLOD 2.0) and stored with its SHA-256 checksum and the operator's "last updated" timestamp. Of its 267 stations we kept the 40 nearest the centroid of all station coordinates; their docking capacities range from 10 to 42 (median 19). Capacity is recorded for context only, because our model has no inventory.

Road matrix. OSMnx downloaded the drive network within a 0.01° buffer of the sandbox's bounding box. The 40 stations snap to 38 distinct road nodes: two pairs of neighbouring stations share a node, which is the source of the two zero-time pairs reported in Section 4.2. Figure B1 shows the geography and both matrices. Directional asymmetry is concentrated in small differences (median 17 s) but has a long tail, with a 90th percentile of 60 s and a maximum of 178 s, typically where one-way streets force a loop. The nine occupied H3 cells hold between one and seven candidate stations, so single-cell pools range from a single station to a substantial cluster.

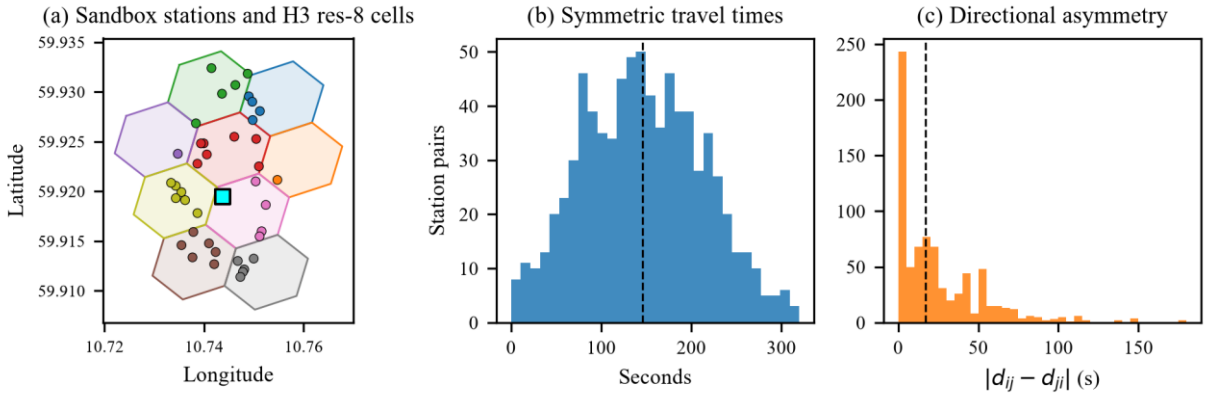


Figure B1. (a) The 39 candidate stations (circles, coloured by H3 resolution-8 cell) and the depot (square). (b) Distribution of symmetrised travel times over all 780 station pairs; dashed line is the median of 146 s. (c) Absolute difference between the two directed travel times before symmetrisation; dashed line is the median of 17 s.

Generator mechanics. Three details of the generator are not visible from the main text. (i) The anchor station is drawn from a random generator seeded only by the instance seed, so each of the five seeds fixes one anchor that is shared by all regimes, sizes, service scales and budgets; seeds therefore act as five spatial neighbourhoods, and the factorial design varies the other factors around them. (ii) Nearest-neighbour and farthest-point selection use planar distance in latitude–longitude degrees, not road time; this is only a selection heuristic, and every route is still evaluated with the road matrix. (iii) The outlier is the selected station whose nearest selected neighbour is farthest away, and only its reward is quadrupled, not its service time. Figure B2 shows the four regimes for the same seed and size.

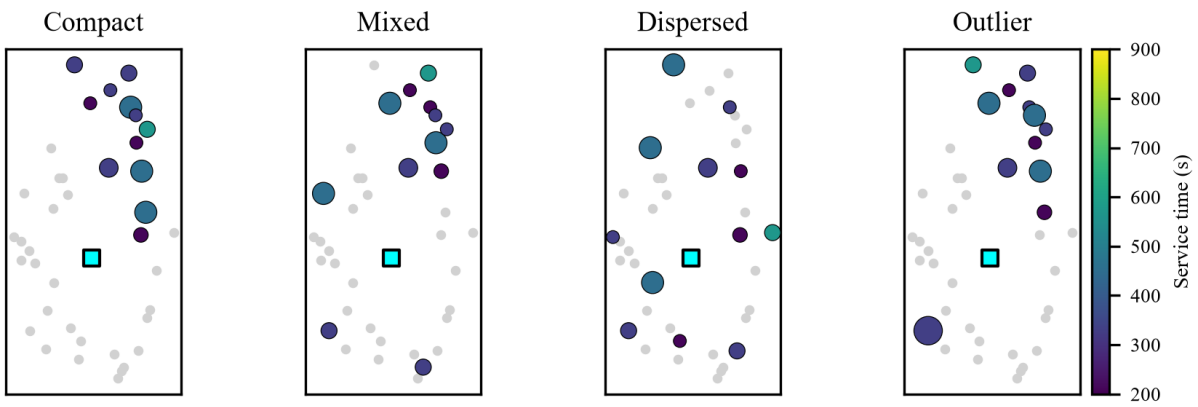


Figure B2. The four spatial regimes for seed 1, n = 12, σ = 1 and a medium budget. Grey dots are unselected sandbox stations; marker size is reward and colour is service time. The outlier panel shows the large-reward isolated station in the south-west.

Task-state distributions. Across all instance–station pairs, task counts of 1, 2, 3, 4 and 5 occurred 30.6%, 31.7%, 18.9%, 16.1% and 2.8% of the time. These shares differ from the clipped Poisson(2) probabilities because the five seeds are reused across the factorial cells, so the sample is far less independent than its size suggests. Service times range from 105 s (one task, σ = 0.5) to 1,380 s (five tasks, σ = 2), and rewards from 1 to 48, the maximum being an outlier station with four tasks at urgency 3. The individual-feasibility floor on T never had to be applied: every budget computed from the full-route fraction already admitted at least one station.

Table B1. Instance profile by factor. Visit share is the fraction of the n stations visited by the optimal route; service share is the fraction of the optimal route's time spent on service.

Group	Mean service (min)	Mean reward	Mean budget (min)	OPT visit share	OPT service share	OPT unused (s)
σ = 0.5	3.0	5.2	25.5	39.6%	65.4%	53
σ = 1	6.1	5.2	43.8	40.7%	77.9%	78
σ = 2	12.2	5.2	80.3	41.5%	87.2%	156
Tight	7.1	5.2	34.9	26.6%	74.5%	93
Medium	7.1	5.2	49.9	40.3%	77.3%	89
Loose	7.1	5.2	64.8	54.9%	78.8%	105
n = 8	7.1	5.8	34.7	37.6%	71.9%	108
n = 12	7.1	4.9	50.0	41.8%	77.5%	91
n = 16	7.1	5.0	64.9	42.5%	81.2%	88

Table B1 makes two properties of the suite explicit. Service dominates the optimal routes, taking 65% of route time even at σ = 0.5 and 87% at σ = 2, which is a consequence of the compact sandbox. And the budget tiers work as intended: the optimal route visits about a quarter of the stations under tight budgets and over half under loose ones, so selection is always a genuine decision.

Instance files. Each instance is a single JSON file holding the instance identifier; depot identifier and coordinates; the budget in seconds; the full $(n+1) \times (n+1)$ integer travel-time submatrix; a list of stations, each with identifier, coordinates, reward, service time, H3 cell and generator metadata (source station identifier, name, capacity, task count, urgency and seed); and instance metadata recording the matrix identifier, source matrix indices, regime, service scale, budget tier and fraction, full-route time, seed, H3 resolution, reward model and the flag that task states are synthetic. A manifest lists all 540 files, and the solver reads nothing else.

Appendix C — Additional Results

Table C1. Quality by service scale and budget tier (180 instances per row). Min is the worst ALG/OPT; match is the exact-reward match rate.

Group	Density min	Insertion min	Split-only mean	Split-only min	Split-only match	S+C min	S+C match
σ = 0.5	0.750	0.600	0.998	0.917	94.4%	0.969	96.7%
σ = 1	0.750	0.778	0.997	0.964	89.4%	0.964	93.3%
σ = 2	0.667	0.833	0.999	0.971	94.4%	0.973	95.6%
Tight	0.667	0.600	0.998	0.917	93.9%	0.973	98.9%
Medium	0.812	0.833	0.998	0.964	90.6%	0.964	92.8%
Loose	0.818	0.897	0.999	0.969	93.9%	0.969	93.9%

Split-only's mean ratio stays between 0.997 and 0.999 in every service-scale and budget group (Table C1), so its quality does not hinge on one part of the design. Tight budgets are where the baselines fail worst (0.600 and 0.667) and where compact pools help most, lifting the match rate from 93.9% to 98.9%. Medium budgets are the hardest tier for both split heuristics. Figure C1 refines this by regime and budget: cheapest insertion falls to 51% matches on compact instances with medium budgets, while split-only stays between 89% and 96% in every one of the twelve cells.

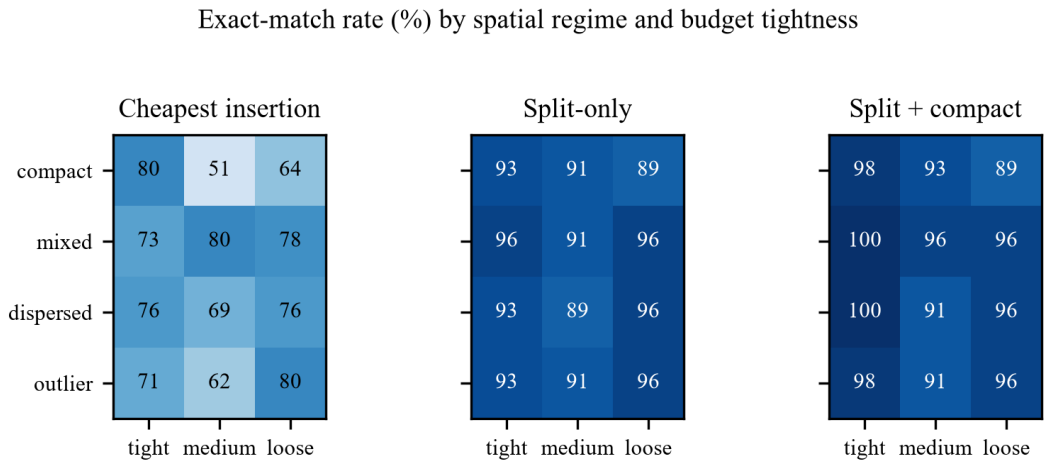


Figure C1. Exact-reward match rate (%) for each spatial regime and budget tier (45 instances per cell).

Figure C2 looks at how routes fail rather than how often. Panel (a) shows that the split heuristics leave nearly as little of the budget unused as the optimum (median 3.0% and 2.9% of T, against 2.7% for OPT and 3.7% for density-then-route). Panel (b) shows the characteristic baseline error: density-then-route visited more stations than OPT on 158 instances, and on 114 of them still collected less reward, because it filled the shift with many cheap, low-value jobs. Split-only did this on only 8 instances.

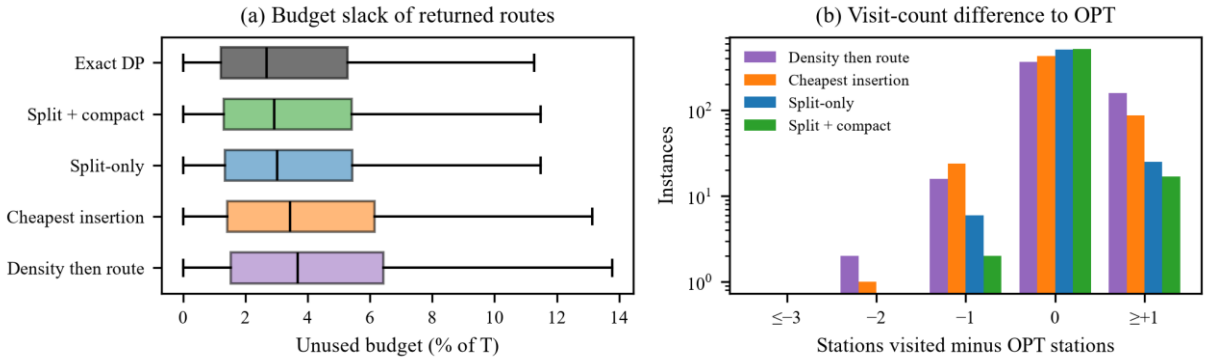


Figure C2. (a) Unused budget as a percentage of T for each algorithm (whiskers span the data without outliers). (b) Number of stations visited minus the number in the optimal route, log scale.

Table C2. Agreement with the exact solver's route over 540 instances. Columns 3–5 split the reward-optimal instances by how the route relates to the exact solver's.

Algorithm	Reward-optimal	Same subset	Same route (up to reversal)	Same subset, other order	Other optimal subset
Density then route	287	217	179	38	70
Cheapest insertion	387	315	259	56	72
Split-only	501	460	385	75	41
Split + compact	514	485	406	79	29

Table C3. Structure of the exact solutions by size.

n	Median feasible subsets	Mean optimal subsets	Mean stations in OPT
8	103	1.75	3.0
12	1,647	2.16	5.0
16	26,451	3.48	6.8

Tables C2 and C3 put the exact-match figures in context. The number of feasible subsets grows roughly sixteen-fold per four added stations, yet the number of optimal subsets stays small, so a heuristic must still locate one of very few optimal subsets among tens of thousands of feasible ones. Split-only's 501 reward-optimal routes use the exact solver's own subset on 460 instances.

Appendix D — Alpha Traces

Figure D1 shows split-only and split plus compactness across the α grid on six instances chosen by rule from the traces: the case study of Section 6.7; the instance whose optimum is reached only at $\alpha \leq 0.45$ with the lowest minimum along the grid; the same for $\alpha \geq 0.70$; the instance with the most distinct reward levels; the instance with a unique best α and the lowest minimum along the grid; and the worst split plus compactness instance. Together they show why no simple search rule over α is safe: optima occur at the low end, the high end or in isolated pockets, and the curves have several local maxima. The compact curve is the maximum over all pools at each α and therefore never lies below split-only.

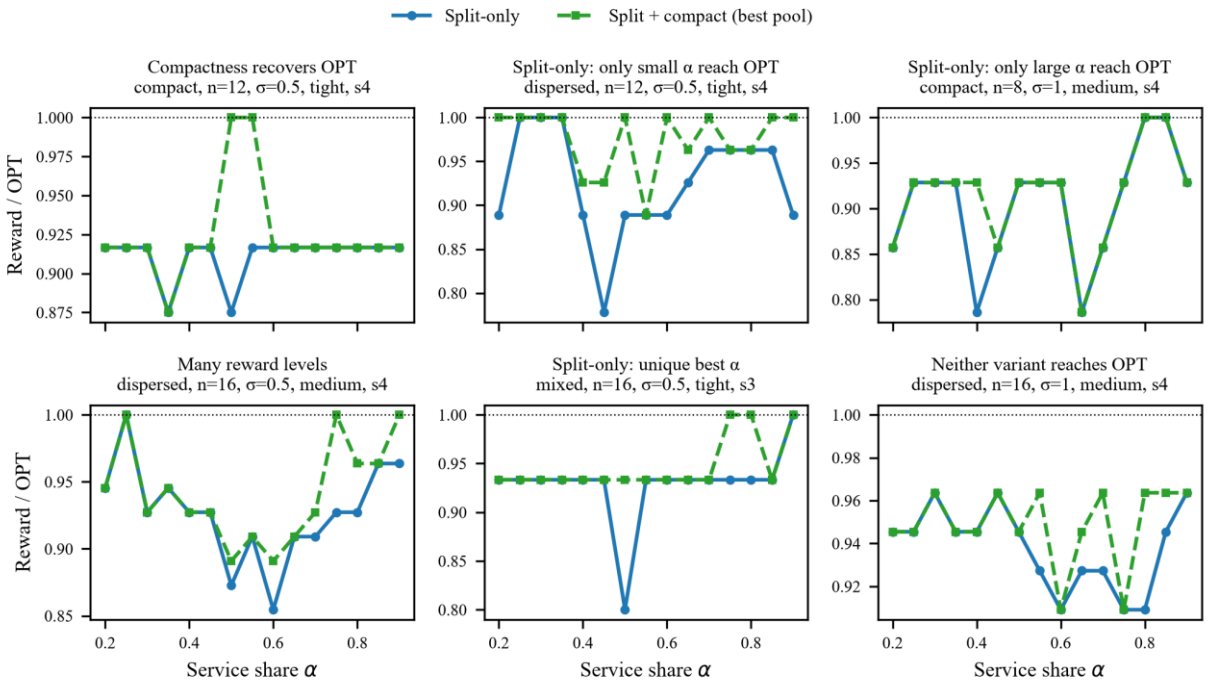


Figure D1. Reward / OPT at each α for six instances (σ is service scale, s is seed). Dotted line marks OPT.

Figure D2 aggregates the traces by factor. Large α values were most often optimal in every group, which is consistent with service dominating route time in this sandbox (Table B1), but the dip around $\alpha = 0.55$–0.70 differs between groups, and loose budgets favour small α almost as much as large α. By regime, outlier instances are distinctive: at $\alpha \leq 0.4$ their mean ratio is the lowest of the four regimes (0.945 at $\alpha = 0.35$), because a small service budget cannot accommodate the quadrupled-reward station, whereas at $\alpha = 0.9$ they are the best (0.992). These shifts are why the grid, not any rule fitted to one group, is the robust choice.

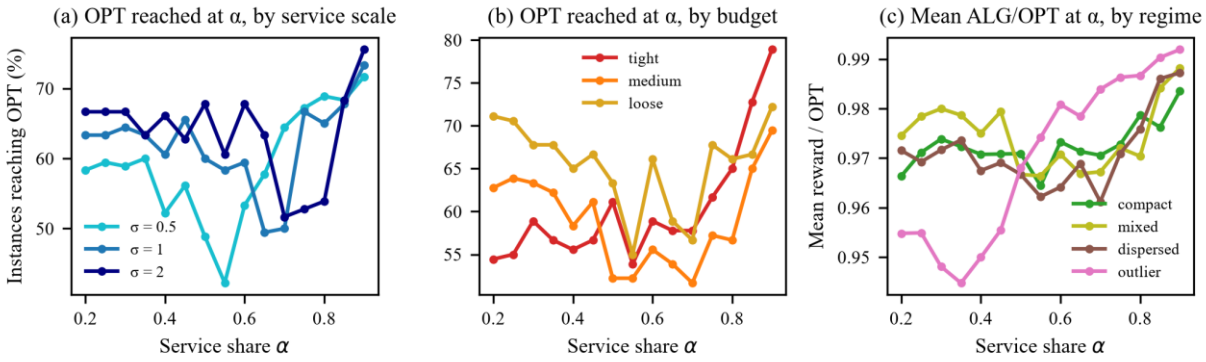


Figure D2. Per-α behaviour of split-only. (a) and (b) Share of instances on which a single α reaches OPT, by service scale and by budget tier. (c) Mean ALG/OPT at each α, by spatial regime.

Figure D3 summarises how much of the grid is redundant. On 93 instances every α gives the same reward, and on a further 255 the grid visits only two reward levels, which explains why most α values tie with the final answer and why the reported chosen α is dominated by tie-breaking. The remaining instances, with three to seven reward levels, are where the grid earns its cost.

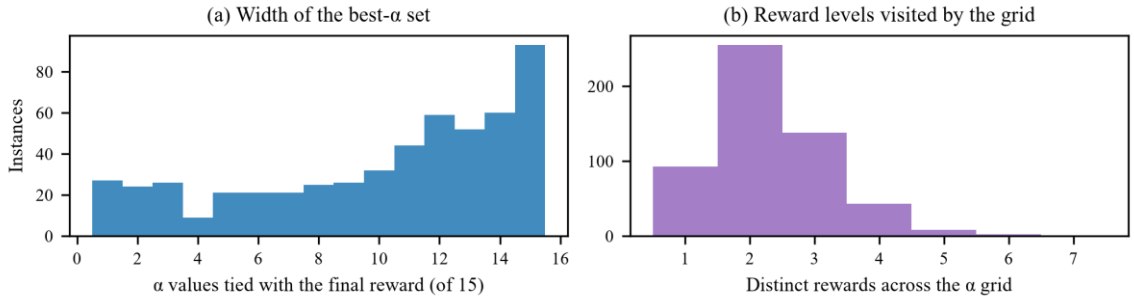


Figure D3. (a) Number of α values (of 15) whose reward equals split-only's final reward. (b) Number of distinct rewards produced across the α grid.

Appendix E — Route Case Studies from the Solution Explorer

The screenshots below were captured from our interactive explorer, which overlays the stored routes of any selected algorithms on a basemap and lists their validated metrics; the captions quote those metrics. Routes are drawn as straight segments between station coordinates with small parallel offsets, so identical routes remain visible; travel times still come from the road matrix. Red is exact DP, blue split-only, green split plus compactness, gold cheapest insertion and magenta density-then-route.



Figure E1. Split-only beats both baselines (dispersed, n = 8, σ = 2, tight). The optimum serves two northern stations for reward 18; split-only finds the same route at α = 0.45. Density-then-route (reward 12) and cheapest insertion (reward 15) commit early to stations in other directions.

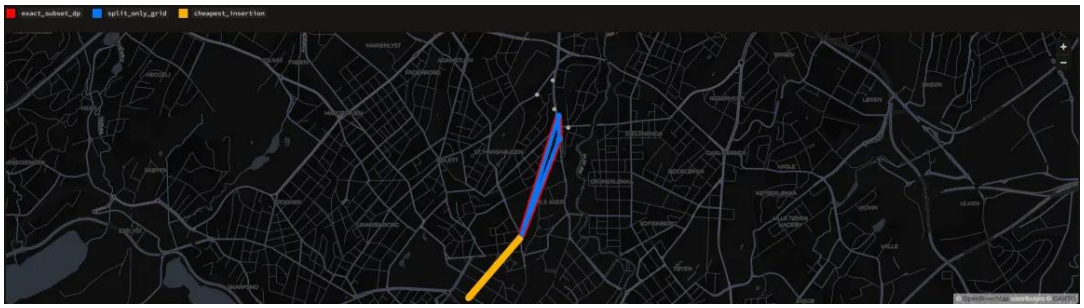


Figure E2. The worst cheapest-insertion instance (mixed, n = 8, σ = 0.5, tight). Insertion greedily takes one high-ratio station to the south-west (reward 9, ratio 0.600) and nothing else fits; split-only at α = 0.25 serves the two northern stations of the optimum (reward 15).



Figure E3. A compactness gain (mixed, n = 12, σ = 0.5, tight). Split-only travels to a distant eastern station and reaches reward 24 (ratio 0.923); the one-ring pool at α = 0.7 keeps the route in a tight triangle near the depot and recovers the optimum of 26.



Figure E4. The worst split plus compactness instance (dispersed, n = 16, σ = 1, medium). Both split variants return the same seven-station route with reward 53 (ratio 0.964), while the optimum swaps in an eastern station for reward 55. No compact pool produced a better route than the global pool, so the compact variant returns the split-only route.

Appendix F — AI Disclosure

AI tools assisted with project planning, implementation and debugging, experiment automation, visualisation, and drafting and editing explanations. The group reviewed the code, result artifacts, mathematical claims, citations and final writing, and takes responsibility for all submitted content.